

Week11 Animation 과제 구현 내용 정리

W11_Team3_Engine / Animation Sequence Viewer, Animation Runtime, Skinning, Crash Dump

1. 전체 구현 개요

이번 과제 구현은 FBX에서 읽은 골격/애니메이션 데이터를 엔진 내부 자산으로 변환하고, 에디터의 Animation Sequence Viewer에서 재생·편집·디버깅하며, 런타임에서는 AnimInstance와 State Machine이 포즈 평가, Notify 발화, 블렌딩을 수행하는 구조로 정리되어 있다. 렌더링 단계에서는 같은 포즈 행렬을 CPU/GPU Skinning 양쪽에서 공유하고 콘솔 명령으로 전환한다.

영역	구현 요약
Import / Asset	FBX AnimStack을 프레임 단위로 베이크하여 UAnimSequence와 UAnimDataModel에 저장. Curve와 Notify도 .sequence 바이너리 자산에 직렬화.
Viewer / Editor	Animation Sequence Viewer, 타임라인, 재생 범위, 프레임 이동, 역재생, Curve/Notify 편집 및 미리보기 Scene 제공.
Runtime Animation	UAnimInstance가 시간 진행, 포즈 샘플링, Notify dispatch, StateMachine transition, pose blending을 담당.
Rendering	USkinnedMeshComponent가 bone global pose와 skinning matrix를 계산. CPU는 정점 버퍼를 갱신하고 GPU는 StructuredBuffer로 행렬을 전달.
Reflection	UCLASS/UPROPERTY 메타데이터와 Generated 코드를 통해 에디터 노출, 직렬화, ObjectFactory 기반 런타임 객체 생성을 연결.
Debug / Crash	Launch 진입점을 SEH로 감싸고 MiniDumpWriteDump와 symbol 기반 crash log를 생성.

2. FBX Animation Import와 AnimSequence 데이터 모델

FBX 임포트는 Scene의 FbxAnimStack을 순회하고, TakeInfo 또는 AnimStack의 TimeSpan에서 시작/종료 시간을 얻는다. FbxTime의 frame rate가 유효하지 않으면 30fps를 기본값으로 사용한 뒤, 각 프레임에서 본 노드의 global transform을 평가한다. 이후 부모 global inverse를 곱해 local transform으로 변환하고 위치/회전/스케일 키를 FRawAnimSequenceTrack에 저장한다.

- 핵심 구조: FRawAnimSequenceTrack은 PosKeys, RotKeys, ScaleKeys를 보관하고, FBoneAnimationTrack은 Bone 이름과 BoneTreeIndex, 실제 트랙 데이터를 묶는다.
- UAnimDataModel은 BoneAnimationTracks, NotifyTracks, FrameRate, NumberOfFrames, NumberOfKeys, CurveData를 가진 실제 데이터 컨테이너다.
- UAnimSequence는 DataModel과 Skeleton/SkeletonAssetPath를 들고 있으며, Viewer와 런타임 재생의 공통 입력 자산이다.
- FbxAnimCurveNode는 재귀적으로 탐색하며 FbxAnimCurve의 key time/value, interpolation, tangent 정보를 FFloatCurve로 변환한다.
- .sequence 저장 포맷은 ASEQ magic/version/header 뒤에 sequence metadata, bone track, float curve, notify track을 바이너리로 기록한다.

주요 파일: JSEngine/Source/Engine/Asset/FbxImporter.cpp, JSEngine/Source/Engine/Animation/AnimData/AnimDataModel.h, AnimTypes.h, AnimSequence.h, JSEngine/Source/Engine/Asset/AnimSequenceAssetLoader.cpp

3. Animation Sequence Viewer와 타임라인

Animation Sequence Viewer는 AnimationSequenceEditorDocument를 중심으로 Sequence, PreviewScene, PlaybackController, Notify/Curve 편집 패널을 연결한다. 화면은 미리보기 뷰포트, 스켈레톤 트리, 시퀀서 타임라인, Notify Details/Debug, Curve Inspection으로 나뉜다. 사용자는 재생/정지, 프레임 단위 이동, 시작/끝 점프, playback range 조절, play rate 변경, 음수 play rate 기반 역재생을 사용할 수 있다.

- FAnimationSequencePlaybackController가 CurrentTime, PlayRate, Looping, PlaybackRangeStart/End를 소유하고 PreviewScene에 pose 적용을 요청한다.
- 타임라인은 시간→화면 좌표 변환, zoom/pan, playhead scrub, playback range handle drag를 제공한다.
- PreviewScene은 USkeletalMeshComponent와 UAnimSingleNodeInstance를 이용해 Viewer 전용 포즈 평가와 Notify 발화를 수행한다.
- Viewer 문서와 위젯은 Notify/Curve 수정 후 .sequence 자산 저장 흐름으로 이어진다.

주요 파일: Editor/Animation/AnimationSequenceEditorDocument.cpp, AnimationSequenceEditorWidget.cpp, AnimationSequencePlaybackController.cpp, AnimationSequenceTimelineWidget.cpp, AnimationSequencePreviewScene.cpp

4. Anim Notify 구현

Notify는 단일 시점 이벤트와 구간형 NotifyState를 모두 지원한다. FAnimNotifyEvent는 StableId, Name, Time, Duration, Color, EventType, NotifyClassName, Payload, TriggerPhase를 가진다. FAnimNotifyTrack은 여러 NotifyEvent를 트랙 단위로 묶어 DataModel 안에 저장된다.

- UAnimInstance는 이전 시간과 새 시간 사이를 검사해 지나간 Notify를 수집한다. 루프, 역재생, playback range wrap까지 고려한다.
- 일반 Notify는 FObjectFactory로 NotifyClassName 객체를 생성한 뒤 Notify() 호출 후 파괴한다.
- NotifyState는 Begin/Tick/End 단계로 분리되며 ActiveNotifyStates 배열에 살아 있는 state 객체를 보관한다.
- 내장 Notify로 SFX, GameplayEvent, CameraShake, FootstepSurfaceEvent, VFX, Decal, Log, AttackWindow 등이 제공된다.
- Payload는 문자열로 저장하지만 에디터에서는 schema 기반 필드 UI와 asset/socket/component picker를 제공한다.

대표 예시: UAnimNotify_PlaySFX는 특정 프레임에서 사운드를 재생하고, UAnimNotify_GameplayEvent는 gameplay event name과 payload를 AnimInstance로 전달한다. UAnimNotifyState_AttackWindow처럼 구간형 Notify는 Begin에서 판정을 열고 Tick 동안 유지하다가 End에서 닫는 방식으로 사용된다.

주요 파일: Engine/Animation/AnimNotify.h/cpp, AnimNotifyBuiltins.cpp, AnimNotifyPayloadParser.cpp, AnimNotifyTypes.h, Editor/Animation/AnimationSequenceNotifyLaneWidget.cpp, AnimationSequenceViewerNotifyDetailsPanel.cpp

5. AnimInstance, Pose 평가, Blending

UAnimInstance는 애니메이션 런타임의 중심이다. CurrentSequence/NextSequence, CurrentTime/NextTime, PlayRate, Looping, BlendFactor를 관리하고, 매 프레임 UpdateAnimation에서 시간 진행과 Notify 처리, State Machine transition을 수행한다. EvaluatePose에서는 현재 시점의 각 bone local transform을 계산해 SkeletalMeshComponent에 전달한다.

- 포즈 평가는 먼저 reference pose로 전체 LocalTransforms를 채우고, 각 FBoneAnimationTrack의 키를 현재 시간에 맞춰 보간한다.
- 위치/스케일은 FVector::Lerp, 회전은 FQuat::Slerp 후 Normalize를 사용한다.
- State transition이 발생하면 NextSequence를 설정하고 BlendFactor를 DeltaTime * BlendSpeed로 증가시킨다.
- BlendPoses는 bone별 translation/scale lerp와 rotation slerp를 수행해 CurrentSequence와 NextSequence 사이를 자연스럽게 연결한다.
- SkeletonAssetPath가 다르면 compatible skeleton remap을 시도하여 track bone name을 target skeleton bone index에 다시 매핑한다.

핵심 흐름: StateMachine Tick → transition consume → Current/Next time advance → Notify collect/dispatch → EvaluatePoseAtTime → BlendPoses → SkeletalMeshComponent ApplyEvaluatedPose

6. Animation State Machine과 Lua 확장

UAnimationStateMachine은 상태, 파라미터, 전이 조건을 엔진 내부 자료구조로 보관한다. 상태는 이름, UAnimSequence, looping, play rate를 가지며, 전이는 From/To state, condition type, parameter name, blend speed, priority를 가진다. Tick에서는 현재 상태 또는 Any 상태에서 출발 가능한 전이를 모으고 priority 기준으로 평가해 첫 성공 전이를 pending transition으로 만든다.

조건 타입	역할
BoolEquals / FloatGreater / FloatLess / BoolNotEqual / FloatNotEqual	Equality state는 Lua script가 갱신한 parameter 값을 기준으로 상태 전환.
Trigger	한 번 소비되는 입력성 이벤트. 예: Jump 트리거.
StateFinished	비루프 애니메이션 종료 후 다음 상태로 전환.
Native	C++ predicate를 등록해 임의 조건 평가.

LuaAnimInstance와 Asset/Script/LuaAnimation.lua, LuaJumpAnimation.lua는 C++ 대신 script에서 state machine parameter를 설정하는 선택 학습 항목까지 연결한다.

7. SkeletalMeshComponent와 Skinning

USkeletalMeshComponent는 런타임 애니메이션을 소유하는 컴포넌트이고, 실제 포즈/스키닝 공통 로직은 USkinnedMeshComponent에 모여 있다. 현재 local pose가 갱신되면 parent-child 계층을 따라 global pose를 만들고, 각 bone의 InverseBindPose * CurrentGlobalPose를 SkinningMatrices로 계산한다.

- CPU Skinning: SkinVerticesCPU가 각 정점의 최대 4개 bone influence를 순회하고 weight를 정규화해 position/normal/tangent를 누적한다. 결과는 SkinnedVertices에 저장되어 정점 버퍼 갱신에 사용된다.
- GPU Skinning: 원본 skeletal vertices를 유지하고 SkinningMatrices를 vertex factory data로 전달한다. HLSL은 StructuredBuffer SkinningMatrices(t16)를 읽어 정점 셰이더에서 동일한 4-weight blend를 수행한다.
- 전환 명령: editor console의 skinning.cpu 0/1 명령이 전역 플래그와 기존 모든 USkinnedMeshComponent 인스턴스를 함께 갱신한다.
- Stat: Animation Update/Evaluate, Bone Transform Update, CPU Skinning, GPU Skinning Instance, Skinned Vertices, Bone Matrix Upload Count 등으로 비교 측정 지점을 제공한다.

주요 파일: Component/SkinnedMeshComponent.cpp, SkeletalMeshComponent.cpp, Render/Proxy/SkeletalMeshRenderProxy.cpp, Render/Mesh/VertexFactory/SkeletalVertexFactory.cpp, Shaders/Common/Skinning.hlsli, Shaders/Material/UberLit.hlsli

8. Bone Weight Heatmap

선택 학습 항목인 Bone Weight Heatmap도 에디터 view mode와 material shader 경로에 들어가 있다. EViewMode::Heatmap이 툴바/컨텍스트 메뉴에 노출되고, 선택 bone index와 bShowSelectedBoneWeight 플래그가 render command debug constants로 전달된다. UberLit shader는 선택 bone의 weight를 HeatmapWeight로 넘긴 뒤 색상 램프로 시각화한다.

9. Reflection System과 Animation 연동

프로젝트의 Unreal 스타일 Reflection은 UCLASS, UPROPERTY, GENERATED_BODY 매크로와 Generated/*generated.h, Reflection.generated.cpp를 통해 클래스/프로퍼티 메타데이터를 생성한다. 이 정보는 에디터 Details 노출, prefab/asset 직렬화 이름, 런타임 객체 생성을 하나로 묶는다.

- USkinnedMeshComponent의 Enable CPU Skinning, USkeletalMeshComponent의 AnimInstance 경로처럼 UPROPERTY가 붙은 값은 에디터에서 수정하고 저장할 수 있다.
- AnimNotify 파생 클래스는 Reflection/ObjectFactory에 등록되어 문자열 NotifyClassName만으로 런타임 객체를 만들 수 있다.
- Animation Sequence Viewer는 Reflection으로 등록된 Notify class 목록과 payload schema를 사용해 사용자가 Notify를 고르고 필드를 편집하게 한다.

- 따라서 Reflection은 Animation 시스템에서 editor UI, asset serialization, notify dispatch를 이어 주는 기반 역할을 한다.

주요 파일: Source/Generated/Reflection.generated.cpp, Source/Generated/AnimNotify*.generated.h, Scripts/GenerateReflection.py, Docs/ReflectionSystemGuide.md

10. MiniDump와 PDB 관계

엔진 Launch는 GuardedMain을 Windows SEH __try/__except로 감싸고, 예외 발생 시 ReportCrash(GetExceptionInformation())를 호출한다. CrashDump.cpp는 Dumps 디렉터리를 만들고, 타임스탬프 기반 .dmp 파일과 .txt crash log를 생성한다. MiniDumpWriteDump는 프로세스, pid, 예외 포인터를 받아 MiniDumpWithDataSegs 형식으로 덤프를 저장한다.

- .dmp 파일은 크래시 당시 메모리/스레드/예외 정보를 담는다.
- .pdb 파일은 주소를 함수명, 파일명, 라인 번호로 해석하는 symbol database다.
- Crash log 생성 시 DbgHelp의 SymInitialize, SymFromAddr, SymGetLineFromAddr64를 사용하므로, 빌드된 exe와 일치하는 pdb가 있을 때 콜스택이 소스 라인까지 해석된다.
- 즉 .dmp만 있어도 예외 분석은 가능하지만, 정확한 함수명과 줄 번호를 보려면 같은 빌드 산출물의 .pdb가 필요하다.

현재 콘솔에는 skinning.cpu 명령은 확인되지만 CauseCrash 등록은 검색 결과에서 확인되지 않았다. CrashDump 저장 시스템은 구현되어 있으므로, 제출 범위에서 CauseCrash까지 요구한다면 EditorConsoleWidget에 의도적 access violation 또는 RaiseException 호출 명령을 추가하면 연결된다.

11. 구현 파일 빠른 색인

주제	대표 파일
FBX import / sequence binary	Engine/Asset/FbxImporter.cpp, Engine/Asset/AnimSequenceAssetLoader.cpp
Anim data model	Engine/Animation/AnimData/AnimDataModel.h, AnimSequence.h, AnimTypes.h, AnimNotifyTypes.h
Anim runtime	Engine/Animation/AnimInstance.cpp, AnimSingleNodeInstance.cpp, AnimationStateMachine.cpp
Notify classes	Engine/Animation/AnimNotify*.cpp, Editor/Animation/AnimationSequenceViewerNotify*.cpp
Reflection	Source/Generated/Reflection.generated.cpp, Source/Generated/*.generated.h, Scripts/GenerateReflection.py
Viewer	Editor/Animation/AnimationSequenceEditor*, AnimationSequenceTimelineWidget.cpp, AnimationSequencePlaybackController.cpp
Skinning / render	Engine/Component/SkinnedMeshComponent.cpp, Render/Proxy/SkeletalMeshRenderProxy.cpp, Shaders/Common/Skinning.hlsli
Crash dump	Engine/Core/CrashDump.cpp, Engine/Runtime/Launch.cpp